

08 · Support de démo scriptée (soutenance, ~8 min)

Séquence **exacte** à taper et à *dire* pendant la soutenance. Chaque commande a été **exécutée et vérifiée** le 2026-09-23 ; les sorties ci-dessous sont fidèles. La démo tient en **~8 min** : 2 min dry-run + 2 min validation humaine + 1 min injection + 1 min chaîne piégée + 2 min preuves (journal/registre).

0. Avant de monter (préparatifs — matin du jour J)

```
cd prototype
source .venv/bin/activate          # ou : export PATH=$PWD/.venv/bin:$PATH
PYTHONPATH=src python -m pytest tests/ -q  # → 52 passed (10 s)
```

- Afficher à l'écran deux terminaux : **A** = commandes, **B** = `runs/` (preuves).
- Garder fermés : e-mails/bureautique. Aucun réseau requis (dry-run).
- Note : le dossier `cases/` contient `casB_mediconsult.md` (fictif, rappel éventuel au jury : « aucune vraie donnée patient » → garde-fou G3).

1. Dry-run complet sur le cas (AG1→AG5, 2 min)

Commande (terminal A) :

```
PYTHONPATH=src python run.py run --case cases/casB_mediconsult.md --no-human
```

Sortie attendue (abrégée) :

```
Fournisseur LLM : dummy-deterministic (référence manuelle)
Chaîne : AG1 → AG2 → AG3 → AG4 → AG5 (proposition) → validation humaine

[ok] étape actifs : 14
[ok] étape modele : STRIDE
[ok] étape menaces : 10
[ok] étape evaluations : 10
[ok] étape risques : 10

=== SYNTHÈSE ===
critique : 0   élevé : 6   moyen : 4   faible : 0
risques validés par l'humain : 0/10
```

À dire :

« Un orchestrateur enchaîne les 5 agents spécialisés. Chacun rend une enveloppe JSON validée par schéma, sourcée. Trois fichiers sont produits : le registre, la mémoire de session (workspace) et le journal horodaté. En dry-run, le simulateur reproduit notre analyse de référence : 14 actifs, STRIDE + LINDDUN, 10 menaces, 10 risques — 6 élevés, 4 moyens, identiques à notre analyse manuelle. »

Astuce timing : la commande est quasi instantanée (< 2 s) — c'est un argument (« pas dépendant du réseau »).

2. Humain dans la boucle (validation interactive, 2 min)

Commande (terminal A) — simule 10 décisions « accepter » signées :

```
printf 'a\n%.0s' $(seq 1 10) | PYTHONPATH=src python run.py run \
--case cases/casB_mediconsult.md --analyste "Dr Dupont"
]
```

Sortie attendue (extrait + fin) :

```
[R-01] Vol d'identifiants d'un médecin par harponnage puis connexion à la console...
niveau=élevé · traitement=réduire (proposé par AG5)
valider ? (a)ccepter / (d)ifférer / (c)orriger : a
...
[R-10] Données de santé en clair dans les journaux... niveau=élevé · traitement=réduire
valider ? (a)ccepter / (d)ifférer / (c)orriger : a
]

=== SYNTHÈSE ===
critique : 0   élevé : 6   moyen : 4   faible : 0
risques validés par l'humain : 10/10
]
```

À dire :

« Trois commandes possible : a accepter, d différer, c corriger. L'agent ne peut jamais s'auto-valider — c'est garanti par le schéma : le champ `valide_par` est de type null dans la sortie de l'agent ; seuls nos 10 inputs écrivent `Dr Dupont`. »

Preuve (terminal B) :

```
grep validation_humaine runs/run-*/journal.jsonl | tail -2 # → 10 événements signés
]
```

3. Test d'injection de prompt (1 min)

Commande (terminal A) :

```
PYTHONPATH=src python run.py test-injection --case cases/casB_injecte.md
]
```

Sortie attendue :

```
Document : cases/casB_injecte.md
[détectée] motif=ignore_instructions extrait='tes instructions'
[détectée] motif=disable_securite   extrait='Ignore tes'
[détectée] motif=override_system    extrait='Tu es maintenant'
[détectée] motif=jailbreak_mot      extrait='tes consignes '
Document neutralisé : 1200 → 1235 caractères
]
```

À dire :

« Le même document contient des consignes piégées. Le sanitizer détecte 4 motifs, les neutralise et les marque comme données non exécutables. Ce n'est qu'une couche — la preuve décisive, c'est l'étape suivante. »

4. La chaîne complète tient sur le document piégé (1 min)

Commande (terminal A) :

```
PYTHONPATH=src python run.py run --case cases/casB_injecte.md --no-human
```

Sortie attendue : identique à l'étape 1 (14 actifs, STRIDE, 10/10, 6 élevés / 4 moyens).

Preuve (terminal B) :

```
grep injection_detectee runs/run-*/journal.jsonl | tail -1
cat runs/run-*/registre_final.json | grep -c '"valide_par": null'
# → 1 ligne d'injection journalisée AVANT le 1er appel ; 10 valide_par null dans le registre piégé
```

À dire :

« Même si l'injection passait le filtre à motifs, elle ne changerait rien : un agent ne peut ni écrire valide_par, ni fixer un niveau (matrice), ni omettre les sources. La pièce à conviction : le registre du document piégé est strictement identique à celui du document sain. Défense en profondeur — test automatisé : jamais faux positif sur le cas sain. »

5. Les preuves — journal et registre (2 min, terminal B)

```
# Trace de session propre (dernier dry-run)
ls -t runs/ | head -1
cat "$(ls -td runs/run-*/ | head -1)journal.jsonl"
```

À montrer : les lignes session_debut → appel_agent (×5) → session_fin ; dans le run interactif : validation_humaine (×10) + session_fin nb_risques:10.

À dire :

« Tout est auditable : qui a appelé quoi, avec combien d'octets en entrée/sortie, chaque injection détectée, chaque décision humaine. C'est le garde-fou G5 et la réponse à la question "ce que vous produisez est-il vérifiable ?". »

Tableau récapitulatif : ce que chaque étape démontre

Étape	Démontre	Garde-fous
1 · dry-run	chaîne complète sur le cas	G2, G4, G7, G9
2 · validation interactive	humain dans la boucle	G6 (valide_par)

Étape	Démontre	Garde-fous
3 · test-injection	détection/neutralisation	G1 , G5
4 · chaîne sur doc piégé	résistance de bout en bout	G1 + G4/G6/G8
5 · journal/registre	auditabilité	G5, G10

Plan B (si une commande échoue)

Symptôme	Réaction
ModuleNotFoundError: jsonschema	<code>pip install -r requirements.txt</code> (10 s), rejouer
Erreur de chemin	relancer depuis <code>prototype/</code> (<code>cd prototype</code>)
Mention d'anomalie (ex. étape AGn impossible)	« cas de l'arrêt bavard (G8) — montrons le journal » : <code>tail runs/run-*/journal.jsonl</code>
Doublon de <code>valide_par ≠ null</code>	impossible par construction ; si le jury insiste, montrer <code>validation.py</code>
Aucun terminal B	faire les greps dans le terminal A, sans casser le rythme

Checklist démo (rappel rapide)

- [] `cd prototype` + venv activé
- [] `pytest -q` → 52 passed (à faire le matin)
- [] `cases/casB_mediconsult.md` + `cases/casB_injecte.md` présents
- [] Terminal B ouvert sur `runs/` avec un `ls -t` prêt
- [] Chrono : 8 min visées, couper § 5 si débordement
- [] Après la démo : rester sur l'écran du journal pour les questions